Universidade Federal de Sergipe — **PROCC**
Projeto e Análise de Algoritmos · Período 2026.2
Módulo II — Otimização, Grafos e Processamento de Cadeias

**Atividade 2 (AV2) — Relatório Técnico**

# Recuperação e seleção otimizada de contexto para IA generativa em materiais educacionais abertos

Programação dinâmica e mochila 0/1 para seleção de chunks sob restrição de tokens: um estudo comparativo com estratégia gulosa

João Cosme Sena Sá · Eduardo Henrique do Lago Silva · Helena Carvalho Leal
Hernandison da Silva Bispo · Nadianne Maria dos Santos Galvão · Rafael Takeguma Goto

| | |
|---|---|
| **Tema** | 8 — Materiais educacionais abertos |
| **Corpus** | *Pense Python* (tradução livre da 3ª ed. de *Think Python*) |
| **Repositório** | Repositório GitHub da equipe |
| **Licença do código** | MIT · Commit de referência registrado |
| **Vídeo da atividade** | *FALTA)* |

Documento revisado em 4 de novembro de 2026 · São Cristóvão, SE
Entrega dos artefatos até 04/11/2026 · Apresentação em 05/11/2026

# 1. Identificação da equipe

A AV2 é desenvolvida por seis discentes do Programa de Pós-Graduação em Ciência da Computação (PROCC/UFS), na disciplina Projeto e Análise de Algoritmos, período 2026.2.

As responsabilidades abrangem especificação formal (Parte A), implementação e corretude (Parte B), análise assintótica (Parte C), experimentação (Parte D) e relação com IA generativa (Parte E).

A distribuição individual está registrada em `docs/CONTRIBUICOES.md` e resumida na Seção 17, com distinção entre responsabilidades acordadas e evidências de execução. A revisão de código, das provas e dos resultados é uma atividade coletiva.

# 2. Tema, corpus e fonte

O trabalho aborda o Tema 8 do enunciado: **materiais educacionais abertos**. O corpus é o livro *Pense Python*, tradução livre da terceira edição de *Think Python*, de Allen B. Downey, realizada por Rodrigo Castelan Carlson [7].

O recorte contém 21 notebooks Jupyter, incluindo prefácio, introdução e capítulos 1 a 19. A aquisição foi registrada em 02/09/2026, utilizando o commit `cfde2c450bf4f2ea65326da88e9968f400597e92`.

O material está disponível em `https://rodrigocarlson.github.io/PensePython3ed/`, com licença CC BY-NC-SA 4.0. O corpus não contém dados sensíveis.

## 2.1 Continuidade da Atividade 1

A AV2 preserva o corpus, a pontuação TF-IDF e a configuração de recuperação C3 da AV1, composta por busca linear e Merge Sort.

A nova etapa seleciona um subconjunto de chunks sob orçamento de unidades lexicais, modelado como mochila 0/1. A programação dinâmica constitui a abordagem principal, comparada a uma estratégia gulosa por relevância/custo [2].

São reutilizadas 30 consultas definidas na AV1, armazenadas em `data/queries.csv`, garantindo a comparação sobre o mesmo conjunto de perguntas.

O texto integral do corpus não é redistribuído no repositório. O script `scripts/baixar_corpus.py` permite reproduzir a aquisição no commit fixado, enquanto `data/corpus_manifest.csv` registra os hashes SHA-256 para verificação de integridade.

# 3. Repositório, licença e reprodutibilidade

O código-fonte está disponível no repositório público:

> Repositório público da equipe (GitHub)

O código utiliza licença MIT (`LICENSE`), distinta da licença CC BY-NC-SA 4.0 do corpus.

A versão de referência consultada para a elaboração deste relatório corresponde à branch `av2-parte-e-rag`, no commit:

> `cfe609c21631db6f585b39b51cf8df166d6f07c1`

# 4. Especificação do problema

## 4.1 Problema e representação dos candidatos

A AV2 investiga a seleção de um subconjunto de chunks recuperados pela configuração lexical C3 da AV1, buscando maximizar a relevância acumulada sob um orçamento de unidades lexicais. O problema é modelado como mochila 0/1, em que cada chunk é selecionado integralmente ou descartado [2].

O corpus corresponde ao livro *Pense Python*, tradução brasileira da terceira edição de *Think Python* [7]. O recorte da AV1 contém 21 notebooks Jupyter, segmentados em 2.737 chunks, dos quais 1.405 são de texto Markdown e 1.332 de código.

Cada candidato $c_i$ possui identificador, texto, custo lexical $w_i$, relevância $v_i$ e metadados, incluindo `source_order`. O custo é obtido de `n_tokens`, que representa unidades lexicais e não necessariamente tokens de um modelo de linguagem.

Para uma consulta $q$, os candidatos são definidos por:

$$C_q = \{c_1, c_2, \ldots, c_n\}, \tag{1}$$

em que $n = |C_q|$ é a quantidade de candidatos com relevância positiva, $w_i \in \mathbb{N}^+$ e $v_i \in \mathbb{R}_{\geq 0}$, com valores finitos. O orçamento máximo é $W \in \mathbb{N}_0$.

A entrada é representada por:

$$I = (q, C_q, W). \tag{2}$$

## 4.2 Relevância lexical

A pontuação dos candidatos preserva o mecanismo TF-IDF da AV1, fundamentado em frequência de termos e frequência inversa de documento [5].

Para uma consulta $q$ e um chunk $c$, a relevância é calculada por:

$$\text{score}(q, c) = \sum_{t \in \text{TermosDistintos}(q)} qtf_t \, \text{tf}(t, c) \, \text{idf}(t), \tag{3}$$

em que $qtf_t$ é a frequência do termo na consulta, $\text{tf}(t, c)$ sua frequência ponderada no chunk e $\text{idf}(t)$ sua frequência inversa documental.

A AV2 mantém a pontuação original, definindo:

$$v_i = \text{score}(q, c_i). \tag{4}$$

A implementação da Parte B converte esses escores em valores inteiros por $v_i^{(q)} = \text{round}(1000 v_i)$. Consequentemente, a otimalidade computacional refere-se aos valores quantizados, e não necessariamente à soma exata dos escores originais. Essa distinção é considerada na análise de corretude e nas recorrências.

A pontuação TF-IDF constitui uma estimativa lexical e não garante, isoladamente, cobertura semântica ou ausência de redundância.

## 4.3 Função objetivo e restrições

A saída é um subconjunto $S \subseteq C_q$, acompanhado do custo lexical total e da relevância acumulada:

$$W_S = \sum_{c_i \in S} w_i, \qquad V_S = \sum_{c_i \in S} v_i. \tag{5}$$

Define-se uma variável binária $x_i$, igual a 1 quando o candidato $c_i$ é selecionado e 0 caso contrário.

A função objetivo da especificação é:

$$\max V_S = \max \sum_{i=1}^{n} v_i x_i. \tag{6}$$

Sujeita às restrições:

$$\sum_{i=1}^{n} w_i x_i \leq W, \tag{7}$$

$$x_i \in \{0, 1\}, \quad i = 1, \ldots, n. \tag{8}$$

O modelo impede a seleção fracionária e a repetição de candidatos. A restrição de capacidade utiliza a mesma unidade lexical para $w_i$ e $W$.

## 4.4 Corretude e critérios de desempate

A corretude da seleção considera duas propriedades: viabilidade e otimalidade em relação à função objetivo adotada.

A viabilidade exige:

$$S \subseteq C_q \quad \text{e} \quad \sum_{c_i \in S} w_i \leq W. \tag{9}$$

Para a formulação original, uma solução ótima $S^*$ satisfaz:

$$\sum_{c_i \in S^*} v_i \geq \sum_{c_i \in S'} v_i, \quad \forall S' \in \mathcal{F}, \tag{10}$$

em que $\mathcal{F}$ é o conjunto de subconjuntos viáveis.

Na implementação com programação dinâmica, a garantia de otimalidade se aplica à soma dos valores quantizados $v_i^{(q)}$. Entre soluções de igual valor quantizado, prioriza-se o menor custo total e, persistindo o empate, a menor sequência lexicográfica de `source_order`.

Casos como candidatos vazios, orçamento nulo, itens maiores que $W$, identificadores duplicados e relevância zero são definidos nos contratos da AV2 e verificados por testes.

### 4.5 Estratégia gulosa e protocolo comparativo

Como linha de base, utiliza-se uma estratégia gulosa que ordena os candidatos pela razão entre relevância e custo lexical:

$$r_i = \frac{v_i}{w_i}. \tag{11}$$

Na implementação, a razão utiliza os valores quantizados $v_i^{(q)}$. Os candidatos são examinados em ordem decrescente de densidade e selecionados quando cabem no orçamento restante. Empates são resolvidos por maior valor e, posteriormente, menor `source_order`.

Embora produza soluções viáveis, essa estratégia não garante otimalidade global para a mochila 0/1.

A comparação utilizou os mesmos candidatos recuperados pela configuração C3 da AV1, preservando corpus, consultas, custos, escores e orçamento entre as estratégias.

Foram avaliadas 30 consultas, $n \in \{10, 30, 50\}$ candidatos e $W \in \{256, 512, 1024\}$ unidades lexicais. A grade experimental compreendeu 270 cenários, com cinco estratégias de seleção e cinco repetições cronometradas por cenário e estratégia, totalizando 6.750 medições.

As métricas incluíram relevância acumulada, utilização do orçamento, tempo de seleção, consumo de memória rastreada e razão de aproximação em relação ao ótimo quantizado. Os procedimentos e resultados são detalhados nas seções de metodologia e resultados experimentais.

## 5. Representação dos dados e estratégia de chunking

A AV2 preserva o processamento do corpus da AV1, mantendo a normalização, a segmentação e a representação lexical dos chunks. A recuperação utiliza a configuração C3 (busca linear e Merge Sort), sem alterações nos critérios de pontuação e ordenação.

### 5.1 Normalização e segmentação

Nas células Markdown, aplica-se normalização NFKC, conversão para minúsculas, tratamento da pontuação e compactação de espaços, preservando os acentos. As células de código mantêm seu conteúdo e indentação originais, com representação lexical separada. Na configuração principal, as stopwords não são removidas e as saídas de execução são descartadas.

O chunking respeita a hierarquia de notebooks, capítulos e seções, sem combinar células Markdown e código. A configuração principal utiliza tamanho máximo de 256 unidades lexicais e sobreposição de 32, produzindo 2.737 chunks. As configurações 128/16 e 512/64 foram avaliadas na AV1, mas não são adotadas como padrão da AV2.

### 5.2 Representação computacional

Cada candidato é representado pelo par $(w_i, v_i)$, acompanhado de identificador, `source_order` e metadados de origem. O valor $v_i$ corresponde à relevância TF-IDF para a consulta, enquanto $w_i$ é obtido do campo `n_tokens`.

O orçamento $W$ limita a soma dos custos dos chunks selecionados. A contagem lexical utilizada como custo é uma aproximação, não necessariamente equivalente aos tokens de um modelo de

linguagem.

A utilização de um único livro limita a generalização dos resultados. Além disso, a relevância lexical pode ser influenciada por termos técnicos em inglês, e a soma dos escores não modela diretamente redundância ou cobertura semântica.

## 6. Algoritmos e pseudocódigo

A recuperação lexical C3 da AV1 produz os candidatos ranqueados. Na AV2, a seleção considera o custo lexical $w_i$ e o valor quantizado $\hat{v}_i = \text{round}(1000 v_i)$ de cada chunk, sob orçamento $W$. Comparam-se a programação dinâmica (DP) e o guloso por densidade.

### 6.1 Programação dinâmica: mochila 0/1

A função `mochila_dp()` ordena os itens por `source_order` e constrói uma tabela de estados para a mochila 0/1. Para representar o desempate por menor custo, utiliza-se a chave $a_i = \hat{v}_i(W+1) - w_i$.

```
DP(C, W):
    validar e ordenar C por source_order
    E[n, w] <- 0, para 0 <= w <= W
    para i <- n-1 ate 0:
        para w <- 0 ate W:
            E[i, w] <- E[i+1, w]
            se peso[i] <= w:
                incluir <- a[i] + E[i+1, w-peso[i]]
                E[i, w] <- max(E[i, w], incluir)
    reconstruir solucao a partir de E[0, W]
    retornar chunks selecionados
```

A tabela considera as alternativas de incluir ou excluir cada candidato. Na reconstrução, empates de valor e custo são resolvidos pela ordem lexicográfica de `source_order`.

### 6.2 Estratégia gulosa por relevância/custo

A função `guloso_densidade()` utiliza a razão $\hat{v}_i / w_i$, selecionando cada candidato integralmente quando seu custo não excede o orçamento restante.

```
GULOSO(C, W):
    validar candidatos e orcamento
    ordenar C por densidade decrescente
    S <- conjunto vazio
    restante <- W
    para cada item em C:
        se peso(item) <= restante:
            adicionar item a S
            restante <- restante - peso(item)
    ordenar S por source_order
    retornar S
```

Em empates de densidade, prioriza-se o maior valor quantizado e, depois, o menor `source_order`. O método é viável, mas não assegura otimalidade global para a mochila 0/1.

### 6.3 Justificativa de corretude

**Programação dinâmica.** Para cada estado $(i, w)$, a tabela armazena a melhor solução obtida com os candidatos restantes e capacidade $w$. Por indução retroativa em $i$, a solução ótima resulta do máximo entre excluir o item e incluí-lo, quando viável. A condição inicial para nenhum candidato restante possui valor zero.

A chave $a_i$ preserva a prioridade da soma dos valores quantizados e, em caso de igualdade, favorece o menor custo. A reconstrução determinística resolve os empates remanescentes. Logo, o algoritmo produz uma solução viável e ótima para a função objetivo quantizada.

**Estratégia gulosa.** O invariante do percurso estabelece que o custo acumulado da seleção não excede $W$. Inicialmente, a seleção é vazia; cada inclusão ocorre apenas quando o item cabe no orçamento restante. Portanto, a viabilidade é preservada até o término.

Entretanto, escolhas locais por densidade podem impedir combinações de maior valor total. Assim, a estratégia garante viabilidade e término, mas não otimalidade. Os detalhes de complexidade são apresentados na Seção 7.

## 7. Análise no modelo RAM

Adota-se o modelo RAM, com custo constante para operações elementares sobre valores de tamanho limitado. Sejam $n$ o número de candidatos, $W$ o orçamento lexical e $k \leq n$ a quantidade de itens selecionados.

A programação dinâmica constrói uma tabela com $(n + 1)(W + 1)$ posições. A validação possui custo esperado $O(n)$ e as ordenações possuem limite $O(n \log n)$. Assim, no pior caso:

$$T_{\mathrm{DP}}(n, W) = \Theta(nW + n \log n), \qquad M_{\mathrm{DP}}(n, W) = \Theta(nW). \tag{12}$$

O custo é pseudopolinomial, pois depende do valor numérico de $W$, e não apenas da quantidade de bits de sua representação.

O guloso por densidade ordena os candidatos e realiza uma varredura linear. Seu custo temporal e espacial é limitado por:

$$T_{\mathrm{G}}(n) = O(n \log n), \qquad M_{\mathrm{G}}(n) = \Theta(n). \tag{13}$$

No pipeline completo, a recuperação lexical C3 e o Merge Sort acrescentam o custo de busca e ordenação dos resultados. Sejam $N$ o número de chunks do corpus, $P$ os resultados positivos, $Q$ o tamanho da consulta e $L$ a soma dos comprimentos dos textos. Os limites superiores são:

$$\begin{aligned} T_{\mathrm{total,DP}} = O(&NQ + L + P \log(P + 1) \\ &+ nW + n \log n), \end{aligned} \tag{14}$$

$$T_{\mathrm{total,G}} = O(NQ + L + P \log(P + 1) + n \log n). \tag{15}$$

A preparação inicial das estatísticas documentais não está incluída no custo por consulta.

# 8. Melhor, pior e caso médio

**Programação dinâmica.** O preenchimento da matriz executa exatamente $n(W+1)$ iterações, independentemente dos valores e custos dos candidatos ou da solução selecionada.

Assim, a inicialização e o preenchimento possuem custo $\Theta((n+1)(W+1))$ em todos os casos de entrada válida.

O custo adicional das ordenações depende da disposição dos elementos. Sob o modelo RAM adotado, os limites assintóticos são:

$$\begin{aligned} T_{\mathrm{DP}}^{\mathrm{melhor}}(n, W) &= \Theta(nW), \\ T_{\mathrm{DP}}^{\mathrm{pior}}(n, W) &= \Theta(nW + n \log n). \end{aligned} \tag{16}$$

**Estratégia gulosa.** O percurso de seleção examina todos os $n$ candidatos. Entretanto, as ordenações podem apresentar comportamentos diferentes conforme a disposição das chaves.

Como o Timsort pode explorar sequências favoráveis à ordenação, os limites são:

$$\begin{aligned} T_{\mathrm{G}}^{\mathrm{melhor}}(n) &= \Theta(n), \\ T_{\mathrm{G}}^{\mathrm{pior}}(n) &= \Theta(n \log n). \end{aligned} \tag{17}$$

O melhor caso linear não deve ser presumido para as consultas reais, pois a recuperação C3 ordena os candidatos por relevância, enquanto o guloso os ordena por densidade de relevância/custo.

**Caso médio.** Uma análise probabilística rigorosa requer uma distribuição de entradas, não definida para este corpus. Assim, não se atribui expressão assintótica específica ao caso médio; os tempos observados são analisados na seção de resultados.

# 9. Recorrências

## 9.1 Mochila 0/1

Na implementação da Parte B, o valor computacional de cada item é quantizado pela escala 1000:

$$v_i^{(q)} = \mathrm{round}(1000 v_i),$$

em que $v_i$ é o escore TF-IDF original e `round()` segue o arredondamento do Python.

A chave utilizada pela programação dinâmica é:

$$a_i = v_i^{(q)}(W+1) - w_i. \tag{18}$$

Essa representação prioriza a maior soma dos valores quantizados e, em caso de igualdade, o menor custo. O desempate por sequência lexicográfica de `source_order` ocorre durante a reconstrução.

Seja $E(i, w)$ a maior chave obtida a partir dos candidatos de $i$ até $n-1$, com capacidade $w$.

A condição inicial é $E(n, w) = 0$. A recorrência é:

$$E(i,w) = \begin{cases} E(i+1,w), & w_i > w, \\ \max\{E(i+1,w),\ a_i + E(i+1,w-w_i)\}, & w_i \le w. \end{cases} \tag{19}$$

A implementação calcula cada estado uma única vez. Sua recorrência temporal de preenchimento é:

$$\begin{aligned} C(i,W) &= C(i-1,W) + \Theta(W+1), \\ C(0,W) &= \Theta(W+1). \end{aligned} \tag{20}$$

Por expansão:

$$C(n,W) = \Theta((n+1)(W+1)). \tag{21}$$

A otimalidade da implementação refere-se aos valores quantizados, não necessariamente à soma exata dos escores originais.

## 9.2 Merge Sort

Na configuração C3, o Merge Sort divide os $P$ resultados positivos em duas partes aproximadamente iguais.

Para $P \ge 2$, sua recorrência é:

$$T(P) = T(\lfloor P/2 \rfloor) + T(\lceil P/2 \rceil) + \Theta(P). \tag{22}$$

Com $T(0) = T(1) = \Theta(1)$.

Para $P$ potência de dois:

$$T(P) = 2T(P/2) + \Theta(P).$$

Pelo Teorema Mestre, $a = 2$, $b = 2$ e $f(P) = \Theta(P)$.

Como $P^{\log_b a} = P$, aplica-se o segundo caso:

$$T(P) = \Theta(P \log P), \quad P \ge 2. \tag{23}$$

O resultado também se aplica a tamanhos que não são potências de dois: a recorrência com piso e teto produz profundidade $\Theta(\log P)$ e custo total $\Theta(P)$ por nível, até fatores constantes.

Incluindo as entradas vazias e unitárias, pode-se escrever:

$$T(P) = \Theta(1 + P \log(P+1)).$$

As recorrências complementam a análise no modelo RAM; scripts e verificações estão documentados no repositório.

# 10. Metodologia experimental

## 10.1 Configuração experimental

Os experimentos utilizaram o corpus *Pense Python*, 30 consultas da AV1 e a recuperação C3 (busca linear e Merge Sort), mantida fixa entre as estratégias.

Foram comparados S1 (prefixo do ranking), S2 (guloso por valor), S3 (guloso por densidade), S4 (guloso modificado) e S5 (mochila 0/1 por programação dinâmica). A comparação principal considerou S3 e S5.

A grade utilizou $n \in \{10, 30, 50\}$ candidatos e $W \in \{256, 512, 1024\}$ unidades lexicais. Foram avaliados 270 cenários e cinco estratégias, com cinco repetições cronometradas após dois aquecimentos, totalizando 6.750 medições.

Uma avaliação complementar calculou a diferença de valor e a redundância textual, além de 1.620 medições do fluxo recuperação–conversão–seleção para S3 e S5, com três repetições por configuração.

## 10.2 Métricas e reprodutibilidade

As métricas de qualidade incluíram valor quantizado, relevância TF-IDF, custo lexical, utilização do orçamento, razão de aproximação e diferença absoluta $\Delta V = V_{\mathrm{DP}} - V_{\mathrm{estrategia}}$, calculada sobre valores quantizados. A redundância foi medida pela similaridade de Jaccard entre conjuntos de palavras, com limiar 0,8.

Foram registrados tempo de seleção, tempo do fluxo completo e pico de alocações Python rastreadas pelo `tracemalloc`. A preparação do corpus foi tratada separadamente do tempo de consulta.

As execuções ocorreram no Fedora Linux, com Python 3.12.16. Dados brutos, gráficos, parâmetros, logs e scripts estão em `experimentos/av2/` e `scripts/av2/`. A equivalência dos arquivos de chunks foi confirmada por SHA-256. A suíte de testes e as auditorias das medições originais e complementares foram aprovadas.

# 11. Resultados e gráficos

A avaliação compreendeu 6.750 medições de seleção e 1.620 medições complementares do fluxo completo. Todas as estratégias respeitaram os orçamentos estabelecidos, e as verificações de consistência não identificaram violações nas condições auditadas.

A Tabela 1 apresenta a comparação principal para $n = 50$ candidatos e $W = 1024$ unidades lexicais. A Figura 1, no Apêndice A, apresenta a variação do tempo de seleção com $n$, para $W = 1024$.

Tabela 1: Medianas de tempo e memória e média da razão de aproximação para $n = 50$ e $W = 1024$

| **Métrica** | **S3** | **S5** |
|---|---|---|
| Mediana do tempo (ms) | 0,3030 | 7,2771 |
| Mediana do pico rastreado (KiB) | 7,34 | 854,07 |
| Aproximação média (%) | 97,78 | 100,00 |

Os valores de memória correspondem ao pico de alocações Python rastreadas pelo `tracemalloc`, não à memória residente total do processo.

Nesse cenário, a programação dinâmica (S5) obteve o valor ótimo da função objetivo quantizada.

O guloso por densidade (S3) alcançou, em média, 97,78% desse valor, com menor tempo de seleção e menor consumo de memória rastreada.

A avaliação complementar registrou diferenças de valor ($\Delta V$) entre as estratégias e a programação dinâmica. Não foram detectados pares de chunks com similaridade de Jaccard igual ou superior a 0,8 nos conjuntos selecionados avaliados. Esse resultado se refere à redundância lexical, não à equivalência semântica.

Nas medições do fluxo completo, a recuperação C3 representou uma parcela predominante do tempo de processamento. Assim, as diferenças de custo entre os seletores foram menos expressivas no tempo total do que nas medições isoladas de seleção.

Os resultados completos, por consulta, quantidade de candidatos e orçamento, estão disponíveis em `experimentos/av2/`, incluindo dados brutos, tabelas e gráficos.

## 12. Discussão de escalabilidade

Os resultados experimentais são compatíveis com a análise assintótica apresentada na Seção 7. A programação dinâmica possui custo temporal $\Theta(nW + n \log n)$ e espacial $\Theta(nW)$, enquanto o guloso por densidade apresenta tempo $O(n \log n)$ e espaço $\Theta(n)$, sob as hipóteses do modelo RAM.

Para $n = 50$ e $W = 1024$, a programação dinâmica apresentou mediana do tempo de seleção de 7,2771 ms, contra 0,3030 ms do guloso. O pico de alocações rastreadas foi de 854,07 KiB e 7,34 KiB, respectivamente.

Esse comportamento evidencia o compromisso entre otimalidade e custo computacional. A programação dinâmica maximiza o valor quantizado sob o orçamento, enquanto o guloso obteve aproximação média de 97,78% nesse cenário, com menor custo de seleção.

Entretanto, nas medições do fluxo completo, o custo da recuperação C3 predominou sobre o da seleção. Portanto, as diferenças observadas entre os seletores não devem ser interpretadas como diferenças proporcionais na latência total do sistema.

A extrapolação para conjuntos maiores de candidatos e orçamentos mais elevados exige novos experimentos, especialmente devido à dependência pseudopolinomial da programação dinâmica em relação a $W$.

## 13. Relação com IA generativa e RAG

A abordagem desenvolvida pode integrar a etapa de preparação de contexto em sistemas de geração aumentada por recuperação (*Retrieval-Augmented Generation* – RAG) [6]. Após a recuperação lexical C3, os chunks são selecionados sob um orçamento de unidades lexicais antes da composição do prompt destinado a um LLM.

A programação dinâmica maximiza a relevância quantizada dentro do orçamento. Para $n = 50$ e $W = 1024$, o guloso por densidade alcançou 97,78% do valor ótimo, em média, com menor tempo e memória de seleção. Isso indica um compromisso entre qualidade da função objetivo e custo computacional.

Entretanto, as unidades lexicais utilizadas não equivalem necessariamente aos tokens de um LLM. Além disso, a pontuação TF-IDF não garante relevância semântica, diversidade ou cobertura completa das evidências.

A seleção depende dos candidatos recuperados: fragmentos relevantes ausentes do ranking C3 não

podem ser recuperados pela etapa de otimização. A função objetivo também não modela explicitamente dependências entre chunks.

Como não foram avaliadas respostas geradas por LLMs, os resultados não demonstram melhorias na qualidade das respostas, no custo de inferência ou na latência de geração. Essas questões permanecem como possibilidades de investigação futura.

## 14. Limitações e ameaças à validade

A validade interna depende da manutenção das mesmas consultas, candidatos, escores e orçamentos entre as estratégias. Embora os experimentos tenham utilizado condições controladas e repetições, os tempos medidos podem sofrer influência do sistema operacional, da execução do Python e de outras atividades do ambiente computacional.

Quanto à validade de construto, a pontuação TF-IDF representa relevância lexical, não necessariamente relevância semântica. O orçamento utiliza unidades lexicais, que podem diferir da tokenização de um LLM. Além disso, a função objetivo aditiva não penaliza diretamente a redundância nem representa dependências entre chunks.

A validade externa é limitada pelo uso de um único corpus educacional, 30 consultas, $n \in \{10, 30, 50\}$ e $W \in \{256, 512, 1024\}$. Consequentemente, os resultados não podem ser generalizados automaticamente para outros corpora, orçamentos ou ambientes.

Por fim, a otimalidade da programação dinâmica refere-se aos valores TF-IDF quantizados, não necessariamente aos escores originais. A ausência de avaliação com respostas geradas por LLMs impede conclusões sobre fidelidade factual, qualidade da geração ou redução efetiva de custos de inferência.

## 15. Conclusão

Este trabalho investigou a seleção de chunks para composição de contexto sob orçamento lexical, utilizando programação dinâmica para mochila 0/1 e estratégias gulosas, a partir da recuperação C3 da AV1.

Os experimentos evidenciaram um compromisso entre otimalidade e custo computacional. Para $n = 50$ e $W = 1024$, o guloso por densidade alcançou aproximação média de 97,78% do valor ótimo quantizado, com mediana do tempo de seleção de de 0,3030 ms, frente a 7,2771 ms da programação dinâmica.

A programação dinâmica oferece uma referência ótima para a função objetivo quantizada, mas apresenta custo pseudopolinomial em relação ao orçamento. As medições do fluxo completo também indicaram predominância do custo da recuperação lexical sobre o da seleção.

Conclui-se que o guloso por densidade constitui uma alternativa de menor custo nos cenários avaliados, enquanto a programação dinâmica permite estabelecer a referência de otimalidade. A avaliação de respostas geradas por LLMs, a utilização de tokenização específica e os critérios semânticos de seleção permanecem como trabalhos futuros.

## 16. Declaração de uso de IA generativa

Ferramentas de IA generativa foram utilizadas como apoio à revisão textual e à estruturação metodológica da especificação.

O registro de ferramentas, modelos, finalidades, prompts e sugestões aceitas ou rejeitadas é mantido em `docs/USO_DE_IA.md`.

A equipe permanece responsável pela revisão das formulações matemáticas, validação do código, demonstrações de corretude e interpretação dos resultados. Sugestões geradas por IA não constituem, isoladamente, evidências científicas.

## 17. Contribuição individual

As responsabilidades individuais foram distribuídas da seguinte forma: João (Partes B e D); Eduardo (B, C e D); Helena (D e E); Hernandison (A e E); Nadianne (A e C); Rafael (B e E).

As evidências de execução, incluindo commits, arquivos, revisões e resultados, são documentadas em `docs/CONTRIBUICOES.md`. A avaliação crítica e a revisão do trabalho são responsabilidades coletivas.

## 18. Vídeo da atividade

O vídeo de apresentação da AV2 documentará a implementação, as verificações de corretude e os resultados experimentais.

A URL será registrada neste relatório, no `README.md` e no `VIDEO.md`, após sua publicação.

## Referências


[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.

[2] KLEINBERG, J.; TARDOS, É. *Algorithm Design*. Pearson/Addison-Wesley, 2006.

[3] SKIENA, S. S. *The Algorithm Design Manual*. 3. ed. Springer, 2020.

[4] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.

[5] MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[6] LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 2020. Disponível em: `https://arxiv.org/abs/2005.11401`.

[7] DOWNEY, A. B. *Think Python*. 3. ed. O'Reilly Media, 2024. Tradução brasileira: *Pense Python*, por Rodrigo Castelan Carlson. Disponível em: `https://rodrigocarlson.github.io/PensePython3ed/`.

[8] PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

# A. Gráficos experimentais complementares

Este apêndice apresenta evidências complementares da avaliação das cinco estratégias de seleção. Os gráficos consideram diferentes quantidades de candidatos e orçamentos lexicais.

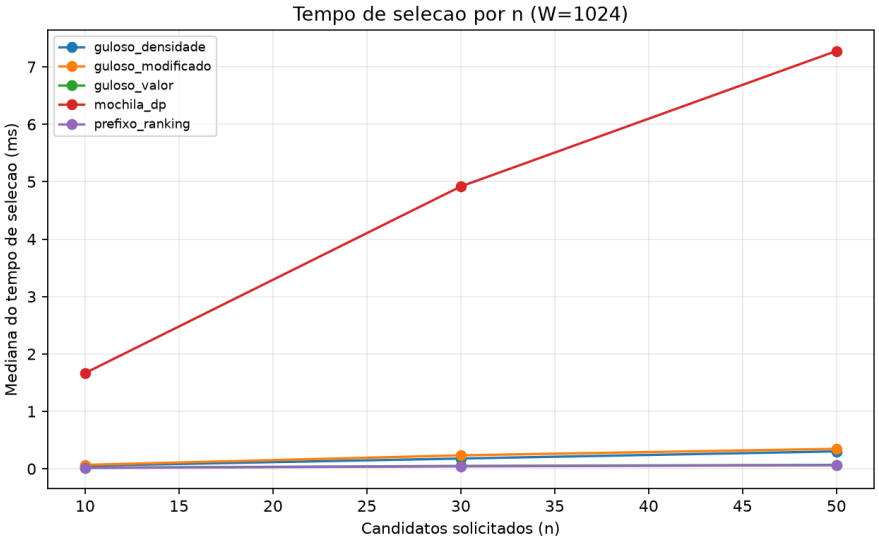



Figura 1: Tempo de seleção em função de $n$, para $W = 1024$.

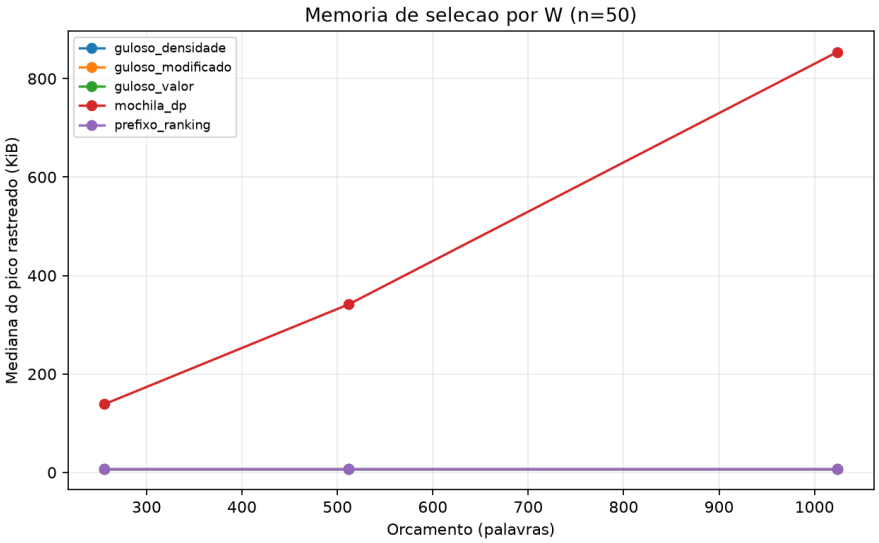



Figura 2: Pico de alocações Python rastreadas em função de $W$, para $n = 50$.

As Figuras 1 a 6, apresentadas no Apêndice A, documentam as análises complementares de tempo, memória, aproximação, diferença de valor, fluxo completo e redundância lexical.

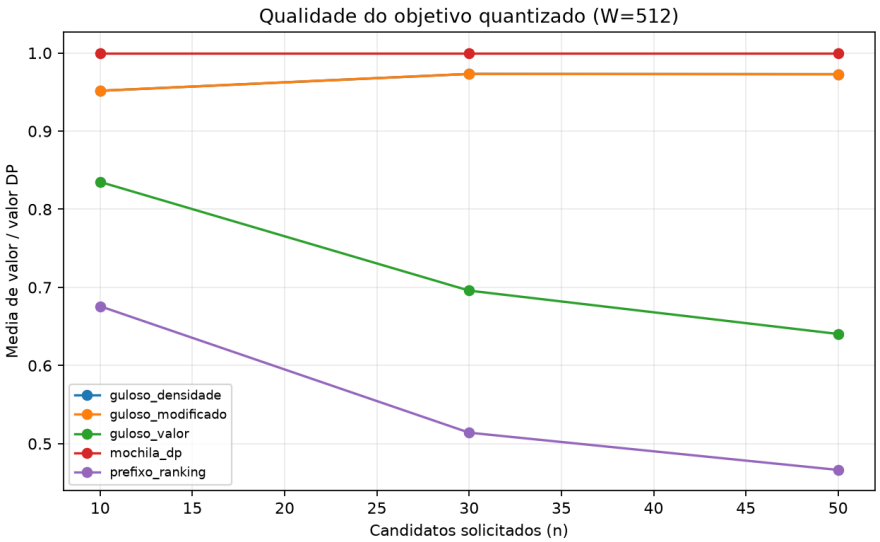



Figura 3: Razão de aproximação em relação ao ótimo quantizado, para $W = 512$.

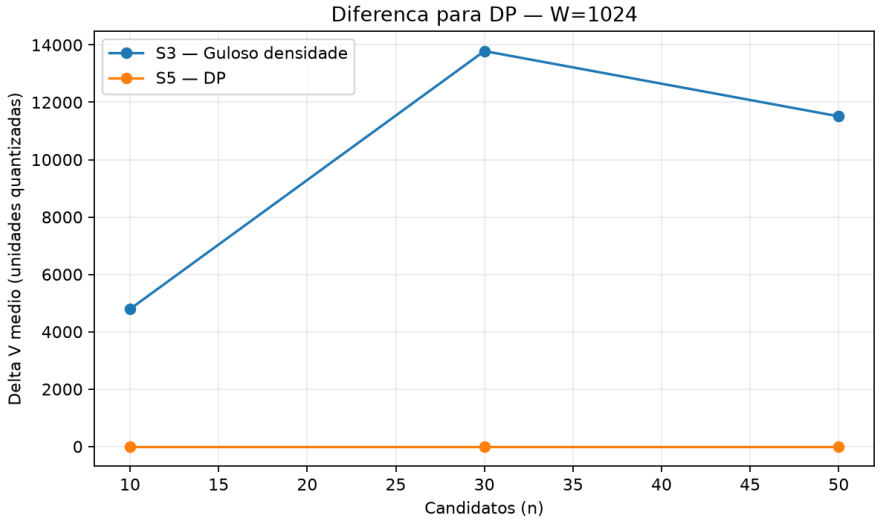



Figura 4: Diferença de valor quantizado em relação à programação dinâmica, para $W = 1024$.

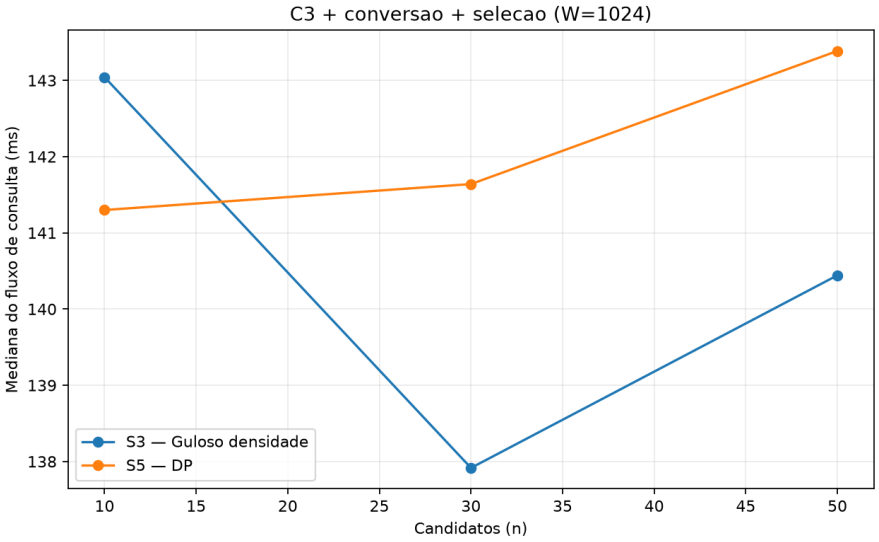



Figura 5: Tempo do fluxo recuperação, conversão e seleção, para $W = 1024$.

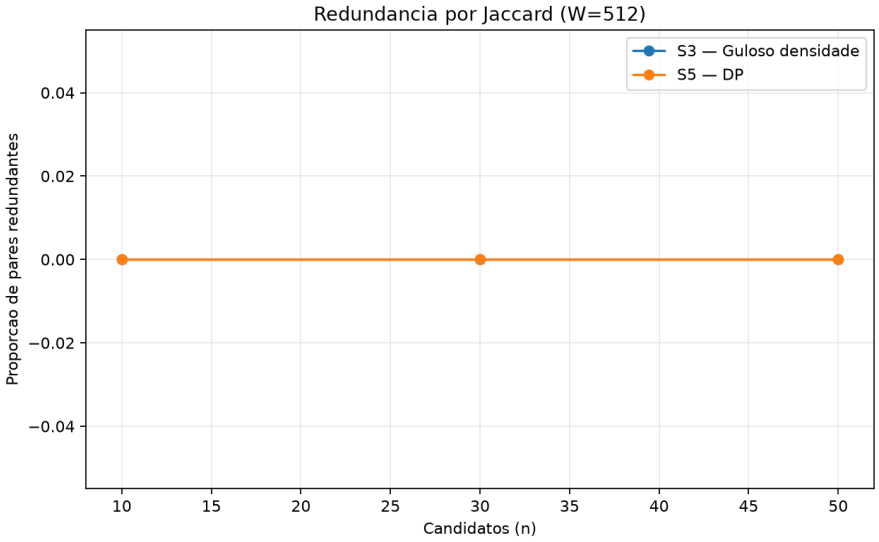



Figura 6: Avaliação de redundância lexical dos chunks selecionados, para $W = 512$.